

05 · Cheatsheet del parcial: fórmulas, reglas y código en una página

Machine Learning · UTB 2026-II · Repaso de 5 minutos antes de entrar

2026-10-09

Fórmulas

Gini de un nodo (con p = proporción de morosos):

$$G = 1 - p^2 - (1 - p)^2 = 2p(1 - p) \in [0, 0.5]$$

$p = 0$ o $p = 1$ (puro) $\Rightarrow G = 0$; $p = 0.5$ (máxima mezcla) $\Rightarrow G = 0.5$.

Gini ponderado de un corte $n = n_{\text{izq}} + n_{\text{der}}$:

$$G_{\text{part}} = \frac{n_{\text{izq}}}{n} G_{\text{izq}} + \frac{n_{\text{der}}}{n} G_{\text{der}} \quad \text{ganancia} = G_{\text{raíz}} - G_{\text{part}}$$

Gana el corte con **menor** G_{part} . El umbral de sklearn es el **punto medio** entre dos valores consecutivos.

Métricas (positivo = mora = 1; matriz con `labels=[0,1]`, `ravel()` da **VN, FP, FN, VP**):

métrica	fórmula	sklearn
tasa base	$\max(p, 1 - p)$ de la prueba	(a mano)
exactitud	$(VP + VN)/n$	<code>accuracy_score(y_true=..., y_pred=...)</code>
precisión	$VP/(VP + FP)$	<code>precision_score(..., pos_label=1, zero_division=0)</code>
recall	$VP/(VP + FN)$	<code>recall_score(..., pos_label=1)</code>
F1	$2PR/(P + R)$	<code>f1_score(..., pos_label=1)</code>

KNN (distancia euclídea entre el nuevo S y cada cliente C del historial; ganan los k más cercanos por votación):

$$d(S, C) = \sqrt{\sum_i (S_i - C_i)^2} \quad z = \frac{x - \text{media}}{\text{desv}} \quad \text{predict_proba} = \frac{\text{vecinos de la clase}}{k}$$

La media y desviación del escalado se calculan **solo con el entrenamiento** (y con la desviación poblacional, la de sklearn).

Reglas de oro (donde se pierden puntos)

1. **Hoja empatada** (mitad y mitad, p. ej. 1 de 2): predice **0** — es lo que hace scikit-learn y lo dice la actividad.
2. **.ravel()** **desempaqueta VN, FP, FN, VP** (fila por fila de `[[VN, FP], [FN, VP]]`). Confundirlo intercambia todas las métricas.
3. **Siempre labels=[0, 1]** en `confusion_matrix` para fijar el orden.
4. El árbol **no necesita escalar variables** (cada pregunta compara una variable consigo misma); KNN sí (las distancias mezclan escalas).

5. Árbol sin límite = **memoriza** (hojas puras) = **sobreajuste**; con clientes nuevos falla. Brecha entrenamiento-prueba grande = alarma.
6. Para el banco el error caro es el **FN** (prestar a quien no paga) → preferir **recall** sobre precisión.
7. La actividad compara **3 cortes dados**; sklearn prueba **todos** los umbrales: por eso pueden no coincidir (el ejercicio restringe la búsqueda).
8. Tasa base = acertar prediciendo la mayoritaria a ciegas. Modelo que no la supera **no sirve**.
9. **KNN escala, el árbol no**: la distancia de KNN mezcla variables de escalas distintas en una sola fórmula (sin escalar, la de números grandes decide sola); el árbol pregunta una variable a la vez.
10. **KNN con k impar** evita empates de votación (2 clases); si aun así empata, sklearn predice la clase 0. `predict_proba` = proporción de vecinos.
11. El scaler se ajusta (fit) **solo con el entrenamiento** y transforma train y prueba: ajustarlo con todo es *data leakage*.

Código esencial (para copiar del lápiz a Python)

```
# ---- construir y entrenar (argumentos por su nombre) ----
from sklearn.tree import DecisionTreeClassifier
arbol = DecisionTreeClassifier(criterion="gini", splitter="best",
                              max_depth=2, min_samples_leaf=5,
                              random_state=0)
arbol.fit(X_train, y_train) # estructura en arbol.tree_

# ---- usar ----
arbol.predict(X_test) # clases 0/1
arbol.predict_proba(X_test)[: , 1] # probabilidad de la clase 1
# (= proporción de 1 en su hoja)
export_text(arbol, feature_names=V) # reglas tipo 'si... entonces'
plot_tree(arbol, feature_names=V, class_names=[...], filled=True)

# ---- métricas (todas con argumentos por nombre) ----
from sklearn.metrics import (accuracy_score, precision_score,
                              recall_score, f1_score, confusion_matrix)
VN, FP, FN, VP = confusion_matrix(y_true=y_test, y_pred=clase,
                                  labels=[0, 1]).ravel()
recall = recall_score(y_true=y_test, y_pred=clase, pos_label=1)

# ---- bosque ----
from sklearn.ensemble import RandomForestClassifier
bosque = RandomForestClassifier(n_estimators=200, max_features="sqrt",
                               bootstrap=True, oob_score=True,
                               n_jobs=-1, random_state=0)
bosque.fit(X_train, y_train)
bosque.oob_score_ # exactitud 'gratis' con los fuera-de-muestra

# ---- KNN (SIEMPRE escalar; fit del scaler SOLO con train) ----
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler().fit(X_train) # 1) fit con entrenamiento
A_train = scaler.transform(X_train) # 2) transformar train
B_test = scaler.transform(X_test) # 3) transformar prueba
knn = KNeighborsClassifier(n_neighbors=5, weights="uniform",
                          metric="minkowski", p=2).fit(A_train, y_train)
knn.predict(B_test) # votación de los k vecinos
knn.predict_proba(B_test)[: , 1] # proporción de votos clase 1
```

Árbol vs bosque en los argumentos

argumento	árbol	bosque	por qué
n_estimators	no existe	100 (fábrica)	cuántos árboles hay
bootstrap	no existe	True	muestra con reemplazo por árbol
oob_score	no existe	False	evaluación con los out-of-bag
max_features	None (todas)	'sqrt' (sorteadas)	hace los árboles distintos
max_depth,		(heredados)	igual en ambos
min_samples_leaf,			
ccp_alpha, criterion			

Respuestas conceptuales en una línea

- **¿Por qué Gini?** Mide “qué tan revuelta” está la caja: 0 = pura, 0.5 = mitad y mitad. El árbol busca cortes que dejen cajas lo más puras posible.
- **¿Por qué el bosque funciona?** Cada árbol es débil y distinto (bootstrap + variables sorteadas); el promedio cancela sus errores individuales (baja la varianza).
- **¿Cuándo árbol, cuándo bosque?** Árbol para *explicar* (comité, reglas); bosque para *acertar* (0.765 → 0.831 en el laboratorio).
- **¿Qué es OOB?** Los ~37 % de clientes que un árbol no vio en su bootstrap: una prueba gratuita cuando no hay conjunto de prueba.
- **¿Por qué KNN escala y el árbol no?** La distancia de KNN suma restas de variables distintas en una misma fórmula: sin escalar, la variable con números grandes decide sola. El árbol compara cada variable consigo misma (“¿ingreso 6.55?”): las escalas nunca se mezclan.
- **¿Qué hace .fit en KNN?** Memorizar los datos (*lazy*): no aprende parámetros; todo el costo está en .predict (distancias a todos).
- **Efecto de k:** $k = 1$ memoriza ruido (sobreajuste); k enorme vota media cartera (subajuste); impar evita empates en 2 clases.
- **Paper (Kinect):** bosque = promedio de las distribuciones de las hojas de T árboles; entrena con ganancia de información (entropía) sobre candidatos aleatorios; con pocos datos, árboles profundos sobreajustan (prof. ~17 con 15k imágenes), con muchos datos siguen mejorando.

Las 10 versiones: respuestas rápidas

Corte ganador de la raíz (sobre los 3 candidatos) y ganancia:

Ver.	p raíz	G raíz	corte ganador	G_{part}	ganancia
V01	0.500	0.500	ingreso 3.5	0.286	0.214
V02	0.400	0.480	razon_deuda 0.50	0.317	0.163
V03	0.400	0.480	ingreso 3.5	0.419	0.061
V04	0.400	0.480	razon_deuda 0.50	0.171	0.309
V05	0.400	0.480	ingreso 3.5	0.400	0.080
V06	0.400	0.480	razon_deuda 0.50	0.317	0.163
V07	0.500	0.500	ingreso 3.5	0.320	0.180
V08	0.400	0.480	razon_deuda 0.50	0.171	0.309
V09	0.400	0.480	ingreso 3.5	0.267	0.213
V10	0.400	0.480	razon_deuda 0.50	0.160	0.320

Métricas finales en la prueba (positivo = mora; generadas con el solver del documento 01, regla “empate $\rightarrow$ 0” incluida):

Ver.	S1	S2	VP	FN	FP	VN	base	exact.	prec.	recall	F1	¿su- pera base?
V01	1	0	1	2	0	5	0.625	0.750	1.000	0.333	0.500	sí
V02	0	0	1	1	1	5	0.750	0.750	0.500	0.500	0.500	empa- ta
V03	1	0	1	1	1	5	0.750	0.750	0.500	0.500	0.500	empa- ta
V04	0	1	1	1	0	6	0.750	0.875	1.000	0.500	0.667	sí
V05	1	0	2	0	1	5	0.750	0.875	0.667	1.000	0.800	sí
V06	1	0	1	1	2	4	0.750	0.625	0.333	0.500	0.400	NO
V07	0	0	1	2	1	4	0.625	0.625	0.500	0.333	0.400	empa- ta
V08	0	0	2	0	2	4	0.750	0.750	0.500	1.000	0.667	empa- ta
V09	1	0	1	1	0	6	0.750	0.875	1.000	0.500	0.667	sí
V10	0	0	2	1	0	5	0.625	0.875	1.000	0.667	0.800	sí

Números del laboratorio (cartera_curso, 300/100)

- Gini raíz **0.4982**; primer corte **ingreso 6.55**; G_{part} **0.3803**; ganancia **0.1179**.
- Árbol **max_depth=2**: 4 hojas; exactitud **0.900**, precisión **0.898**, recall **0.930**, F1 **0.914**; matriz 37/6/4/53.
- De fábrica: 75 hojas, 0.980 entrena / 0.770 prueba (brecha **+0.210**).
- Árbol vs bosque (20 repartos): **0.765** vs **0.831**, desviación 0.037.
- KNN k=5 escalado: exactitud **0.880** (precisión/recall/F1 **0.895**), matriz **37/6/6/51**; mejor k de la lista: **k=7 con 0.890**; k=1 (sobreajuste): **0.760**.